perf

菜单 UI 性能检测指南

演示弹窗菜单 · 显示 / 交互 / 稳定性测量

1. 概述与指标
2. 测试场景与按键注入
3. 显示性能测量
4. 交互性能测量
5. 代码热点与内存检查
6. 执行流程
7. 结果记录模板
8. 附录　命令速查

1. 概述与指标

1.1 适用对象

本指南针对 `com.skyworth.storesystem` 演示弹窗左侧菜单，核心对象为 `MenuListLayout`、`BaseDialog` 与 `MarketBaseItemView`。 菜单由代码直接创建的 View 树承载，**不是 RecyclerView**，性能主要取决于 焦点切换时的 measure / layout / draw 成本，以及滚动时的 smoothScroll 帧率。

菜单运行在视频播放之上的 Dialog 中，**基线负载应为“视频播放中打开菜单”**。 静态页面下的数据只能作为参考，不能代表门店实际场景。

1.2 指标与阈值

阈值按 1080p / 60Hz 设备给出，单帧预算 16.7 ms。若目标设备 UI 刷新率为 50Hz，按 20 ms 重新换算。 测量时固定同一台设备、分辨率与刷新率，避免跨设备混比。

维度	指标	建议阈值
打开	菜单键按下 → 面板首帧上屏	≤ 100 ms；View 池命中 ≤ 50 ms
焦点移动	按键 → 焦点行重绘完成	P90 ≤ 16.7 ms，P99 ≤ 33 ms
流畅度	Janky frames 占比	≤ 5%，无 > 50 ms 帧
丢键	连按 20 次方向键后的焦点移动次数	= 20，不漏键、不跳项
静置	无输入 10 s 内的新增帧数	0 帧
稳定性	反复开关 30 次后的 PSS 增长	< 2 MB，Dialog 实例数不增长

1.3 测量纪律

- 使用 `assembleNo_iconDebug`；release 包会剥离日志，且调试信息不完整。
- 固定同一台设备、分辨率、刷新率与片源；记录设备型号、系统版本与提交号。
- 每个场景先 `reset`，连续跑 3 次取中位数；改动前后各存一份原始数据。
- 计时只看 trace 或帧数据，不拿 `adb` 命令的返回时刻当应用耗时。

2. 测试场景与按键注入

2.1 显示场景

- 2 项（面板自适应高度）、3 项（正好不滚动）、4 项以上（列表滚动）。
- 超长标题（触发跑马灯）、4 行副标题（面板最高）。
- 菜单叠在播放中的视频上，作为真实基线。
- 冷启动首次打开（View 池为空）与第二次打开（池命中）对比。

2.2 交互场景

- 逐项移动，覆盖枚举、开关、复合开关、进入项与点击项。
- 首项再按上 / 末项再按下（双击收起）；单次按下不应收起。
- 左右切换枚举项、确定键切换开关。
- 无间隔连按 20 次方向键，以及长按连发（key repeat）。

2.3 稳定性场景

- 反复开关菜单 30 次。
- 不同类型演示交替打开，验证 `MarketItemViewPool` 命中与 Context 失效丢弃行为。
- 打开期间切后台、旋转、切换视频源。

2.4 按键注入

`adb shell input keyevent` 通过 InputManager 注入按键，事件会像真实遥控器一样 进入当前焦点窗口，因此可用于完整的焦点与点击路径。每次调用都会在设备端启动一个 `app_process`，注入速率低于真实长按连发；它只负责产生事件，**不包含应用渲染耗时**。

```
# KEYCODE_MENU 打开 / 收起菜单
adb shell input keyevent 82

# 逐项下移 10 次（带间隔，用于测单帧成本）
for ($i=0; $i -lt 10; $i++) { adb shell input keyevent 20; Start-Sleep -Milliseconds 150 }

# 快速连按 20 次（无间隔，用于测丢键与突发负载）
for ($i=0; $i -lt 20; $i++) { adb shell input keyevent 20 }
```

按键	keycode	按键	keycode
菜单	82	确定	23
上 / 下	19 / 20	左 / 右	21 / 22
返回	4	主页	3

抓取内容详解

`input keyevent` 没有可分析的输出，本节的“抓取内容”指的是它注入的事件在应用侧留下的轨迹。 按键从 **InputReader → InputDispatcher → 当前焦点窗口 → View.dispatchKeyEvent** 逐级分发： `MenuListLayout.dispatchKeyEvent` 先记录按下 / 抬起，再交给焦点项处理； 方向键移动焦点后，`MarketBaseItemView.onFocusChanged` 触发重绘。用 `logcat` 统计焦点获得次数可以判断是否丢键，用 Perfetto 观察从 `InputDispatcher` 到下一帧 `doFrame` 的间隔才是端到端延迟。

> **注** 无间隔循环并不等同于遥控器长按。真实长按由系统按键重复产生事件，频率与设备配置有关； 需要复现长按时，可重复注入同一按键，并在 trace 中观察应用是否被连续滚动淹没。

3. 显示性能测量

3.1 gfxinfo：帧统计

每个场景开始前清空统计，执行场景后读取。该命令抓取的是**当前进程自 reset 以来** 由应用渲染并提交的帧，不包括其他进程，也不包括被合成器丢弃、应用从未提交的帧。

```
adb shell dumpsys gfxinfo com.skyworth.storesystem reset
# 执行一组场景：打开菜单、移动焦点、切换开关
adb shell dumpsys gfxinfo com.skyworth.storesystem
```

抓取内容详解

字段	含义与判读
`Total frames rendered`	自 reset 起应用提交的总帧数。菜单静置时该值应停止增长；若持续增长，说明有常驻重绘。
`Janky frames`	超过帧预算的帧数与占比。新系统按 FrameTimeline 统计，数值更贴近实际卡顿。
`Janky frames (legacy)`	旧口径统计，部分 ROM 仍保留；与新口径同时出现时，以新口径为主。
`50th / 90th / 95th / 99th percentile`	帧耗时分布（ms）。P50 是常态，P90 / P99 反映尾延迟。平均值会被少数长帧掩盖，不作为主指标。
`Number Missed Vsync`	未在 vsync 前准备好的帧数。通常说明 UI 线程或 GPU 已经跟不上刷新节奏。
`Number High input latency`	输入事件到达应用时已经偏老。多见于主线程繁忙、事件分发被阻塞。
`Number Slow UI thread`	measure / layout / draw 超出帧预算。菜单焦点变化触发的重布局会记在这里。
`Number Slow bitmap uploads`	位图上传 GPU 超时。背景图、大图标切换时值得关注。
`Number Slow issue draw commands`	RenderThread 下发绘制命令超时，多与复杂图层、阴影、裁剪有关。
`Number Frame deadline missed`	未在截止时间前完成上屏的帧数。新系统的核心卡顿计数。
`HISTOGRAM`	帧耗时直方图，按毫秒分桶。用于看尾部分布，部分系统版本不输出。

判读顺序：先看 P90 / P99 是否越界，再看 Slow 计数指向 UI 线程、位图上传还是绘制命令， 最后结合 trace 定位到具体 View 或阶段。计数是累计值，一个场景对应一次 reset，不要把多个场景混在一起。

> **注** 字段名称与统计口径随 Android 版本和厂商 ROM 略有差异，缺失字段不代表该项为 0； 以设备实际输出为准，跨版本比较时只看同名且同口径的字段。

3.2 framestats：逐帧拆解

`framestats` 在 gfxinfo 的基础上输出最近一段帧的逐帧时间戳，用来判断一帧到底慢在 UI 线程、RenderThread 还是 GPU。

```
adb shell dumpsys gfxinfo com.skyworth.storesystem framestats
```

抓取内容详解

输出先打印统计头，随后出现一行 `---PROFILEDATA---`，其后的 CSV 才是逐帧数据。 每一行是一帧，列顺序如下（时间戳单位为**纳秒**，来自单调时钟）：

列	含义
`FLAGS`	该帧的卡顿原因位掩码，按位与判断，见下方说明。
`IntendedVsync`	该帧原本计划开始的 vsync，作为总耗时基准。
`Vsync`	该帧实际开始的 vsync；被推迟时会晚于 IntendedVsync。
`OldestInputEvent`	本帧处理的最老输入事件时间。
`NewestInputEvent`	本帧处理的最新输入事件时间。
`HandleInputStart`	开始处理输入的时间。
`AnimationStart`	开始执行动画的时间。
`PerformTraversalsStart`	开始 measure / layout 的时间。
`DrawStart`	UI 线程开始 draw 的时间。
`SyncStart`	RenderThread 开始同步绘制树的时间。
`IssueDrawCommandsStart`	开始向 GPU 下发绘制命令的时间。
`SwapBuffers`	交换缓冲、把帧交给合成器的时间。
`FrameCompleted`	该帧完成上屏的时间。

计算方式

- **总帧耗时** = `FrameCompleted - IntendedVsync`；超过 16.7 ms 记为丢帧。

- **输入处理** = `HandleInputStart - IntendedVsync`；偏大说明事件到得太晚。
- **UI 线程** = `DrawStart - HandleInputStart`；没有输入的帧可用 `DrawStart - PerformTraversalsStart` 观察布局与绘制。
- **同步绘制** = `SyncStart - DrawStart`。
- **命令下发** = `IssueDrawCommandsStart - SyncStart`。
- **交换缓冲** = `SwapBuffers - IssueDrawCommandsStart`。
- **上屏等待** = `FrameCompleted - SwapBuffers`。

下面脚本筛选逐帧数据，按总耗时降序输出最慢的 20 帧：

```
adb shell dumpsys gfxinfo com.skyworth.storesystem framestats |
  Select-String -Pattern '^\d+,' |
  ForEach-Object {
    $c = $_.Line.Split(',')
    [pscustomobject]@{
      Flags      = [int]$c[0]
      TotalMs    = ([double]$c[12] - [double]$c[1]) / 1e6
      UIThreadMs = ([double]$c[8]  - [double]$c[5]) / 1e6
    }
  } | Sort-Object TotalMs -Descending | Select-Object -First 20
```

FLAGS 位	含义
1	错过 vsync
2	UI 线程过慢
4	位图上传过慢
8	绘制命令下发过慢
16	资源上传过慢
32	交换缓冲过慢

> **注** `framestats` 只保留最近一段帧（环形缓冲），长场景前先 reset； 时间戳为 0 表示该阶段没有发生（例如无输入帧）。列数可能随系统版本增加， 固定列脚本建议先核对表头，优先按列名取值。

3.3 静置重绘检测

菜单打开但不按任何键时，不应产生新帧。该检查专门捕捉常驻重绘，成本很低，建议每次改动都做。

```
adb shell dumpsys gfxinfo com.skyworth.storesystem reset
Start-Sleep -Seconds 10
adb shell dumpsys gfxinfo com.skyworth.storesystem | Select-String "Total frames"
```

抓取内容详解

对比 reset 前后 `Total frames rendered`：新值为 0 或保持不变才合格。若持续增长， 说明有 View 在无输入时仍然 `invalidate`。优先排查以下来源：

- 焦点标题的跑马灯：`MarketBaseItemView.updateFocus()` 在聚焦时设置 `ellipsize = MARQUEE` 并 `setSelected(true)`，标题超出宽度时 TextView 会持续请求重绘。
- 开关、箭头等子 View 的动画或无限循环动画。
- 加载态、闪烁提示、进度指示器未在空闲时停止。

确认有常驻帧后，用 Perfetto 的 `view` / `gfx` 数据源查看 `Choreographer#doFrame` 的调用来源，再回到对应 View 关闭无效重绘。

3.4 Perfetto：定位到阶段

Perfetto 抓取系统级时间线，可以把“按键 → 输入分发 → measure / layout / draw → 上屏”串成一条链。 trace 文件是二进制 protobuf，不能用文本工具读取，需用 ui.perfetto.dev 或 `trace_processor` 打开。

```
adb shell perfetto -o /data/misc/perfetto-traces/menu.perfetto-trace -t 20s sched freq idl
e am wm gfx view input binder_driver
adb pull /data/misc/perfetto-traces/menu.perfetto-trace
```

抓取内容详解

数据源	抓取内容
`sched`	线程调度：谁在运行、谁在等待、是否被抢占。用于判断 UI 线程是否被其他任务挤占。
`freq`	CPU 频率变化。频率长期偏低时，同样的代码也会变慢。

数据源	抓取内容
`idle`	CPU 空闲状态，判断是否频繁进出低功耗状态。
`am`	Activity 生命周期，用于对齐“打开菜单”的起点。
`wm`	窗口管理：焦点变化、窗口布局、relayout。
`gfx`	帧时间线与 Choreographer / RenderThread 切片，定位帧耗时。
`view`	View 系统的 measure / layout / draw 细节。
`input`	InputReader / InputDispatcher：按键从内核到窗口的完整路径。
`binder_driver`	Binder 事务，排查跨进程等待（例如播放器、系统服务）。

重点查看以下切片：

- `Choreographer#doFrame`：每帧的入口，持续时长即 UI 线程为该帧工作的总时间。
- `measure` / `layout` / `draw`：位于 `performTraversals` 内，定位布局成本。
- `DrawFrame`：RenderThread 的绘制流程，观察同步、命令下发与交换缓冲。
- `InputDispatcher`：按键分发时间，与下一帧 `doFrame` 的间隔就是交互延迟。
- `SurfaceFlinger` / `HWC`：合成与上屏，确认是否卡在合成阶段。

> **注** Perfetto 需要 Android 9 及以上；更老的设备可用 `atrace` 或 Android Studio 的 System Trace。trace 文件体积较大，抓取时长控制在 20 s 左右，并在复现动作前后留出余量。

3.5 打开耗时的定向埋点

在 `BaseDialog.show()`、`MenuListLayout.resetData()` 与首次上屏之间加入临时 `android.os.Trace`，把打开过程拆成“构建 → 测量布局 → 首帧绘制”。Trace 仅用于测试， 验证后移除，不进入正式版本。

```
Trace.beginSection("menu:resetData");
try {
    // resetData 主体
} finally {
    Trace.endSection();
}
```

抓取内容详解

自定义切片会以 `menu:resetData` 的名字出现在抓取进程的主线程轨道上，可直接读到 构建菜单树、更新每一项文本与回收旧 View 的耗时。与 `am` / `wm` / `gfx` 切片对照，可以判断首帧时间花在应用构建，还是花在窗口添加与合成。

另外确认两处延迟是否叠加：`MarketBaseItemView.update()` 为默认焦点安排的 `postDelayed(…, 100)` 请求焦点，以及首次 `setVisibility(VISIBLE)` 中 `postDelayed(…, 30)` 的滚动对齐。

3.6 SurfaceFlinger：上屏延迟

视频播放时 UI 层与视频层同时在场，是最容易掉帧的场景。先找到窗口 layer，再读取该 layer 最近一段缓冲队列的上屏时间戳。

```
adb shell dumpsys SurfaceFlinger --list | findstr /i storesystem
adb shell dumpsys SurfaceFlinger --latency "<上一步找到的 layer>"
```

抓取内容详解

`--list` 列出当前所有 layer 名称，包含包名/Activity 的条目通常是应用的窗口层； 视频、SurfaceView 会有独立 layer，不要选错。`--latency` 按帧输出三列时间戳（单位纳秒）：

列	含义
DesiredPresentTime	该帧计划上屏的 vsync 时间。
ActualPresentTime	该帧实际出现在屏幕上的时间。
FrameReadyTime	应用把缓冲交给合成器的时间。

- **上屏延迟** = `ActualPresentTime - DesiredPresentTime`；为正值表示晚于计划。
- **排队等待** = `DesiredPresentTime - FrameReadyTime`；用于判断缓冲是否过早/过晚入队。
- 行内出现 `0` 或 `9223372036854775807` 表示空槽或未完成的帧，统计时剔除。

输出通常只保留最近一段（约 128 帧）的历史，应在菜单可见、持续按键时抓取，并对差值排序取 P90 / P99。

> **注** Dialog 在多数设备上属于宿主 Activity 的窗口层，不一定有独立 layer 名； 若列表中找不到，优先选择包含当前 Activity 的 layer，避免误选视频层。

4. 交互性能测量

4.1 按键 → 焦点延迟

端到端延迟定义为：按键由 InputDispatcher 投递，到焦点行完成重绘并出现在下一帧。测量方式有两种：

- 在 Perfetto 中量 `InputDispatcher` 切片结束，到焦点行那次 `Choreographer#doFrame` 完成的间隔。
- 临时在 `MenuListLayout.dispatchKeyEvent`（ACTION_DOWN）与 `MarketBaseItemView.onFocusChanged(true)` 打 Trace，差值即分发与视觉更新的成本。

抓取内容详解

Perfetto 中一条按键事件会依次经过 `InputReader`（从设备读取）、 `InputDispatcher`（找到焦点窗口并派发）与应用的输入队列；应用侧在 `dispatchKeyEvent` 中处理，焦点变化后由 Choreographer 安排下一帧。 因此延迟包含“分发 + 事件处理 + measure / layout / draw + 上屏”四段，单看其中一段会低估问题。

4.2 丢键与跳项

焦点获得次数必须等于注入的按键次数。菜单的边界收起逻辑依赖 `mDownKeyEvent / mUpKeyEvent` 字符串比对，突发输入时最容易出现漏键或误收起。

```
adb logcat -c
1..20 | ForEach-Object { adb shell input keyevent 20 }
adb logcat -d -s rightLeftMenuItem | Select-String "gainFocus is true" | Measure-Object
```

抓取内容详解

- `logcat -c` 清空主缓冲区，避免历史日志干扰计数。
- `-d` 打印当前缓冲后立即退出，不持续阻塞。
- `-s rightLeftMenuItem` 只保留该 TAG；菜单项焦点日志由 `L.R_L_MENU_ITEM` 输出，消息中包含 `onFocusChanged() gainFocus is true`。
- `Measure-Object` 统计匹配行数；结果应等于注入次数。若偏少，说明事件被合并、丢弃， 或焦点没有真正切换。

> **注** 该统计只用于判断丢键，不能用于计时：logcat 的缓冲与回读会显著改变时延。 日志级别由 `L` 统一控制，release 包可能不输出，务必使用 debug 包。

4.3 滚动与切换

- **滚动**：4 项以上连续下移，观察 `MenuListLayout.scrollNow()` 的 `smoothScrollTo` 是否被反复重启，造成滚动追逐、抖动或跳变； trace 中检查滚动帧是否连续满帧。
- **枚举 / 开关**：左右切换与确定键切换时，检查是否只重绘变化的子 View， 避免整块面板 relayout；开关的 `setSelected` 只应刷新 drawable 状态，不应触发重新测量。
- **边界收起**：单次按下不应收起，双击才收起；确认误触率与收起动画的帧耗时。

4.4 日志开销

`MenuListLayout.dispatchKeyEvent` 与 `MarketBaseItemView.onFocusChanged` 每次按键都会拼接字符串并调用 `L.d`。日志级别判断发生在方法内部， **字符串拼接发生在调用之前**，即使最终不输出也会产生分配。压测时先确认这部分占比， 避免把日志成本算进交互耗时；必要时临时关闭对应 TAG 再测一次作为对照。

5. 代码热点与内存检查

5.1 代码热点

位置	风险	验证方式
`MarketBaseItemView.updateFocus()`	每次焦点变化取背景 drawable、改文字颜色、切换 marquee	单次焦点移动的 doFrame 耗时；静置帧数
`MenuListLayout.dispatchKeyEvent()`	每键字符串拼接与日志；边界双击判断	快速连按的丢键率；trace 中该方法耗时
`MenuListLayout.resetData()`	全量回收、`removeAllViews`、重新 addView；池上限 5/类型	冷 / 热打开耗时对比；交替打开 30 次后的 inflate 次数与内存
`MenuListLayout.scrollNow()`	每次获焦都 smoothScroll，首次显示还有 30 ms postDelayed	连续下移是否出现滚动追逐或跳变

位置	风险	验证方式
`MarketBaseItemView.update()`	默认焦点 postDelayed 100 ms 后再 requestFocus	从打开到可操作的总时长
`MarketItemViewPool`	Context 不同即丢弃；类型池上限 5	切换不同演示后统计池命中率（临时计数日志）

5.2 内存检查

```
adb shell dumpsys meminfo com.skyworth.storesystem
```

抓取内容详解

区域	关键字段与判读
`App Summary`	`TOTAL PSS` 是总体内存；`Private Dirty` 是应用独占的脏页，最适合判断增长。 Java Heap / Native Heap / Graphics 分别对应托管堆、原生分配与图形缓冲。
`Objects`	`Views` 与 `ViewRootImpl` 是 Dialog / 窗口泄漏的直接信号； 反复开关菜单后应回落到基线。`Activities` 应保持稳定，`AppContexts` 增长说明有 Context 未释放。
`ViewRootImpl`	每个窗口一个。菜单 Dialog 的 ViewRootImpl 在关闭后应被回收， 持续增长即存在窗口或 Dialog 泄漏。

测量步骤：记录一次基线 → 反复开关菜单 30 次 → 退到空闲状态等待 GC → 再取一次快照。比较 `TOTAL PSS`、`Private Dirty`、`Views` 与 `ViewRootImpl`。 内存快照存在分配器噪声，单次波动不作为结论，至少取 3 次中位数。

> **注** `PSS` 会把共享页按比例计入，跨进程比较时意义有限； 判断本应用是否泄漏，优先看 `Private Dirty` 与对象计数。

6. 执行流程

1. 固定设备、分辨率、刷新率与片源，安装 `assembleNo_iconDebug`。

2. 冷启动第一次打开菜单，记录 View 池为空时的首帧耗时。
3. 收起后再次打开，记录池命中时的首帧耗时。
4. 依次覆盖 2 项、3 项、4 项以上、超长标题与 4 行副标题。
5. 视频播放中重复上述打开与移动，作为真实基线。
6. 菜单打开后静置 10 s，确认新增帧数为 0。
7. 逐项移动 10 次，记录 P90 / P99 与 Janky 占比。
8. 无间隔连按 20 次，核对焦点移动次数。
9. 覆盖边界收起、左右切换、开关切换与长按连发。
10. 反复开关 30 次，记录 PSS、Views 与 ViewRootImpl。
11. 保存 gfxinfo 全文与一条 Perfetto trace，按版本归档。

> **注** 改动前后各保留一份原始数据。只有五项固定指标同时不退化，才认为性能没有回退； 单项数值变好但尾延迟变差时，按回退处理。

7. 结果记录模板

指标	改动前	改动后	结论
打开耗时（冷 / 热）	—	—	待填
焦点移动 P90 / P99	—	—	待填
Janky frames 占比	—	—	待填
连按 20 次焦点移动数	—	—	待填
静置 10 s 新增帧数	—	—	待填
开关 30 次 PSS 增长	—	—	待填

记录要点

- 写明设备型号、系统版本、分辨率与刷新率，以及构建变体和提交号。
- 环境数据（gfxinfo 全文、Perfetto trace）作为附件归档，正文只放结论与五项指标。
- 出现回退时同时记录当时打开的演示类型与菜单项数量，便于复现。

- 日志、trace 与内存快照可能包含设备路径，归档前确认不含未脱敏信息。

附录　命令速查

命令	抓取内容	关键字段
`dumpsys gfxinfo <pkg>`	帧统计与卡顿计数	P90 / P99、Janky、Slow UI thread
`dumpsys gfxinfo <pkg> framestats`	逐帧时间戳	FrameCompleted − IntendedVsync
`dumpsys SurfaceFlinger --latency`	缓冲队列上屏时间	ActualPresent − DesiredPresent
`perfetto` / `atrace`	系统级时间线	Choreographer#doFrame、InputDispatcher
`dumpsys meminfo <pkg>`	内存与对象计数	TOTAL PSS、Views、ViewRootImpl
`logcat -d -s <tag>`	应用日志	焦点获得次数、错误与状态迁移
`input keyevent <code>`	注入遥控按键	事件轨迹，不含渲染耗时

本指南针对 com.skyworth.storesystem 演示弹窗菜单。测量前请确认设备已通过 adb devices 连接。